Distillation of SWE Agent Trajectories into Small Models

Maria Khodorchenko
ITMO University
Saint Petersburg, Russia

Vitaly Poberezhsky
ITMO University
Saint Petersburg, Russia

Abstract

Large language model (LLM) agents for software engineering can solve tasks by iterating over code, shell, and file-editing tools inside an executable sandbox, but the resulting trajectories are long, noisy, and expensive to reuse for training smaller models. We study how to distill agent trajectories into a 4B student model under severe context constraints. Using the `Nemotron-SWE-v1` trajectories as our only source of training data, we compare three preprocessing strategies: (i) raw full trajectories, (ii) two heuristic filtering variants that keep only progress-making steps, and (iii) trajectory chunking into prompt-completion samples around each assistant action. On a 1000-trajectory train split and 100-trajectory test split, chunking reduces the mean sequence length from 46.8k tokens to 6.5k tokens and yields the best held-out perplexity (1.336), tool-name accuracy (70.2%), and tool-argument exact match (22.9%). The filtered variants are more conservative, but they retain less useful supervision and remain weaker than chunking on all measured token-level metrics. The results suggest that for SWE-agent distillation, the best compression is not to delete large portions of the trajectory, but to restructure it into compact local decision windows while preserving the surrounding task context. Exact-match metrics remain too strict to measure end-to-end task success, so the next step is to evaluate distilled policies in a sandboxed environment with executable tests.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; *Maintaining software*; • **Computing methodologies** → **Machine learning**.

Keywords

software engineering agents, trajectory distillation, tool use, small language models, supervised fine-tuning

ACM Reference Format:
Maria Khodorchenko and Vitaly Poberezhsky. 2026. Distillation of SWE Agent Trajectories into Small Models. In *Proceedings of the International Workshop on Agentic AI for Next-Generation Software Development (AgenticDev 2026), co-located with ASE 2026.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Agentic software development has rapidly moved from single-turn code completion to multi-step systems that plan, execute tools, observe outputs, and recover from errors inside a sandboxed environment. OpenHands is a representative open platform for this style of interaction: it exposes file editing, command execution, and multi-agent orchestration around executable development tasks [1]. More recently, SWE-Gym introduced a training environment with real Python repositories and unit tests, making it possible to optimize agents against executable software engineering tasks rather than only static trajectory imitation [2]. Parallel to that line of work, agent distillation has shown that small models can inherit tool-using behavior from larger teachers when the training signal is carefully packaged [3, 4].

This paper studies a practical question that appears early in a distillation pipeline: what should be distilled from long SWE-agent trajectories so that a small model can actually learn from them? The answer is not obvious. A raw trace contains system prompts, user requests, thoughts, tool calls, tool outputs, retries, and terminal logs. Much of that content is redundant, and much of it is too long for a 4B student model. At the same time, naive compression can remove the very steps that teach the model how to choose tools and arguments. Our goal is therefore to compare simple, reproducible trajectory transformations and determine which ones preserve the most useful supervision under length limits.

Our study is intentionally narrow. We use only one public dataset, `Nemotron-SWE-v1`, and only one student model family, Qwen3-4B fine-tuned with Unsloth QLoRA. We do not claim a final distillation recipe or a benchmark result. Instead, we report a focused empirical study on trajectory preprocessing and the difficulty of evaluating token-level imitation with strict exact-match metrics.

The main findings are:

- Full SWE trajectories are extremely long for a small student: in our loaded split, the mean trajectory length is 46,801 tokens and the 95th percentile is 75,272 tokens.
- A conservative filtering strategy that keeps only progress-making steps shortens trajectories, but it also weakens the supervision signal and yields only moderate improvements.
- Chunking trajectories into local prompt-completion windows works best among the tested options. It reduces mean length to 6,516 tokens and achieves the best perplexity and exact-match metrics on the held-out set.
- Exact tool-name and argument matching is too strict to measure the real quality of agentic behavior; environment-level evaluation in a sandbox is the right next step.

2 Related Work

OpenHands and executable agents. OpenHands provides an open development platform for generalist AI software developers, emphasizing tool use, sandboxed code execution, and multi-agent coordination [1]. This kind of environment produces the kind of trajectories we want to distill: long, structured, and grounded in actual tool outputs.

Training and evaluating SWE agents. SWE-Gym introduced a training setting with executable Python repositories and unit tests, enabling both policy learning and verifier training from real software engineering episodes [2]. This line of work is especially relevant to our supervisor's suggestion that the final evaluation should move from string matching to task success in a container.

Agent distillation. Recent agent-distillation work shows that tool-using behavior can be transferred into smaller models when the teacher traces are transformed into a useful learning signal. Distilling LLM agents into small models with retrieval and code tools demonstrates that compact models can learn tool-using policies from teacher trajectories [3]. Kimi-Dev further argues that agentless or workflow-style training can serve as a skill prior for SWE agents [4]. Our study differs from both by focusing specifically on preprocessing long SWE trajectories before fine-tuning a small student.

3 Problem Setting

We consider trajectories of the form:

$$\tau = (m_1, m_2, \ldots, m_T),$$

where each message m_t is one of: system instruction, user request, assistant reasoning, assistant tool call, tool result, or final answer. The distillation target is a student model that predicts the next assistant action conditioned on a local prefix of the trajectory.

Two constraints dominate the problem.

First, trajectories are very long. In our analysis of the raw `Nemotron-SWE-v1` training trajectories, the mean length is 46,801 tokens, the median is 43,394 tokens, the maximum is 257,031 tokens, and the 95th percentile is 75,272.2 tokens. Those lengths are far beyond what a 4B model can handle comfortably without aggressive truncation or expensive long-context adaptation.

Second, token-level supervision is sparse and structurally noisy. A trajectory may contain multiple assistant turns, thoughts, shell outputs, and repeated attempts. If we train directly on the full sequence, most of the context is either masked or irrelevant to the immediate action choice. The key design question is therefore how to compress trajectories while preserving the causal supervision that teaches the student to choose the next tool call.

4 Data

We used the `r2e_gym` split of `Nemotron-SWE-v1`. In the loaded split, the dataset contains 51,029 trajectories with the columns `uuid`, `messages`, `license`, `used_in`, `tools`, `dataset`, and `repo`. The message field stores OpenHands-style interactions with system, user, assistant, and tool roles.

For experimentation, we split the data into:

- **Train:** 1000 trajectories
- **Test:** 100 trajectories

The choice was practical rather than benchmark-driven. We wanted a small but diverse slice that could be repeatedly reprocessed during preprocessing experiments, finetuning trials, and metric checks.

5 Preprocessing Strategies

We compared three trajectory representations.

5.1 Raw full trajectories

The most direct option is to keep each trajectory as a single sample. This preserves all interaction history, but it is too long for a small model. In practice, training on the raw format forces heavy truncation, and Unsloth may drop examples whose remaining labels are entirely masked after truncation. In our logs, this behavior was visible as a warning that some samples were removed because no response remained after truncation.

5.2 Progress-making filtering

The second family of methods deletes assistant turns that appear unhelpful. The guiding intuition is that a bad turn should matter only if it leads to future progress. We implemented two variants.

Filter v1 keeps only assistant-to-tool pairs that show local progress, removes all `think` steps, and retains `finish` turns.

Filter v2 adds a more global look-ahead criterion: a step is kept not only when its own tool output looks useful, but also when it contributes to eventual progress in later steps.

Both filters reduce sequence length, but they differ in how aggressively they prune the trajectory. The second version is more selective about which steps deserve to remain in the distilled dataset.

5.3 Chunked prompt-completion windows

The third method restructures each trajectory into multiple prompt-completion samples. Each chunk contains the system prompt, the user request, and a limited window of preceding context, while the completion contains only the current assistant action. This representation preserves local decision-making while avoiding the need to model the entire episode at once.

Empirically, this was the most successful setup. In our data preparation workflow, the chunked train set contained 61,585 samples and the chunked test set contained 6,207 samples. Mean sequence length dropped to 6,516 tokens, the median to 6,002 tokens, the maximum to 16,380 tokens, and the 95th percentile to 11,092.8 tokens.

6 Model and Training

We fine-tuned a Qwen3-4B student with Unsloth using parameter-efficient adaptation (QLoRA). The main practical issue was sequence length. The raw trajectories were so long that simply increasing the context window made training slow and still left the model vulnerable to truncation. This led us to focus on preprocessing rather than long-context scaling.

The chunked representation matched the expected supervised fine-tuning interface more naturally than the full-trajectory format. In our chunked preprocessing pipeline, each sample is split into an explicit prompt-completion pair: the prompt contains the system message, the user request, and a bounded context window, while the completion contains the current assistant action. We set the prompt labels to -100 and compute loss only on the completion tokens. In contrast, the filtered formats stay closer to the original trajectory structure and are trained through the chat template with response-only supervision, so all tokens before the assistant response are masked while assistant turns remain trainable. Even then, the resulting samples can carry long-tail context and remain expensive.

We tracked two classes of metrics.

Training metrics. We report the initial and final training loss over the monitored run and the corresponding reduction percentage.

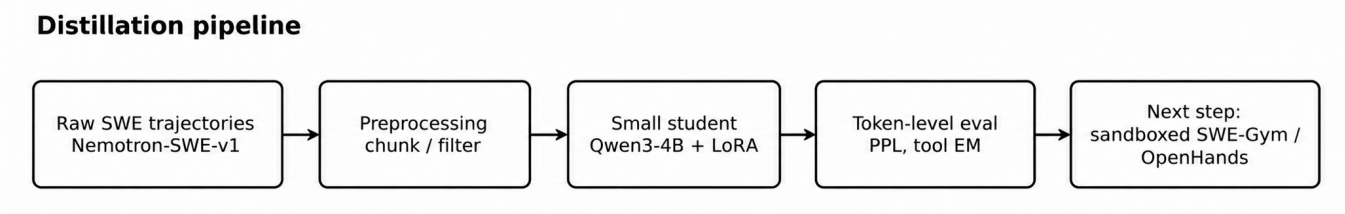


Figure 1: End-to-end distillation pipeline used in the study. Raw SWE trajectories are preprocessed into chunked or filtered views, fine-tuned with a 4B student, and then evaluated with token-level diagnostics before moving to sandboxed task execution.

Held-out token-level metrics. On the test split we compute perplexity and strict exact-match measures:

- **Tool-name accuracy**: exact match on the tool name.
- **Arguments exact match**: exact match on tool-call arguments.
- **Joint exact match**: both name and arguments must match.
- **Finish accuracy**: exact match when the gold action is `finish`.
- **Trajectory success rate**: fraction of trajectories where every evaluated step is correct.

These metrics are deliberately harsh. They are useful for debugging data transformations, but they do not capture semantically equivalent actions, reordered steps, or alternative but valid tools.

Figure 1 summarizes the whole distillation pipeline.

7 Results

Table 1 compares the resulting length distributions, and Table 2 reports the held-out evaluation metrics. The results are consistent across all metrics. Chunking is the strongest representation, filter v2 is slightly better than filter v1, and both filtering variants remain behind chunking by a substantial margin.

The most informative comparison is between chunking and filtering. Chunking reduces the mean sequence length by almost an order of magnitude relative to raw trajectories while also improving held-out perplexity from 1.42–1.45 down to 1.336. At the same time, strict action prediction improves substantially: tool-name accuracy rises from roughly 61–63% to 70.2%, and exact argument matching rises from roughly 17% to 22.9%.

The loss curves tell the same story. Figure 2 visualizes the three training runs side by side.

Chunking starts with a higher loss because the dataset is less redundant and the model must learn more from each sample, but it ends with the largest relative reduction (68.5%). The filtered setups also converge, but not as strongly.

Two findings deserve special attention. First, finish accuracy is 0.0 in all reported test runs. This indicates that the student still struggles with the final "stop and return the answer" decision, even when it learns tool-name patterns. Second, the filtered runs still have 912 truncated steps, which means that length control remains imperfect even after pruning. Chunking eliminates this issue.

8 Discussion

8.1 What seems to work

Our experiments suggest that the best unit of supervision for SWE-agent distillation is not the full trajectory, but the local action window. The student benefits from seeing the surrounding task context, the immediate tool choice, and the exact completion target, but not from the full history of every intermediate attempt. Chunking explicitly exposes that local decision boundary and appears to preserve the most useful signal.

8.2 What seems to be lost by filtering

Filtering is tempting because it removes obviously unhelpful turns. In practice, however, it is difficult to define "unhelpful" without knowing the later outcome. Our two filters showed that retaining only progress-making steps does compress the data, but the resulting supervision is weaker than chunking. This likely happens because some seemingly redundant steps still encode error recovery, tool selection preferences, or the transition from exploration to exploitation.

8.3 Why exact match is not enough

A small model may reorder actions, choose a different but equivalent tool, or express the same intent with a slightly different argument structure. Exact string matching will count those cases as failures even when the trajectory is semantically correct. For that reason, our token-level metrics are only a diagnostic layer. The proper evaluation target is end-to-end task success inside a sandbox with executable tests.

8.4 Next step: sandboxed evaluation

The natural next step is to move the distilled student into an environment such as SWE-Gym or the OpenHands SDK and test it on real tasks. That would answer the more important research questions: which trajectories are actually useful for downstream success, how much compression is safe, and whether the distilled student can still complete tasks after tool execution and feedback loops are introduced. In that setting, trajectory selection could be driven by final task outcome rather than by string similarity alone.

9 Threats to Validity

Our study is limited in several ways.

First, it uses one dataset and one student model family. Different codebases, tool schemas, or model architectures may react differently to the same preprocessing.

Second, our exact-match metrics are harsh by design. They are good for comparing preprocessing variants, but they underestimate semantically correct variation.

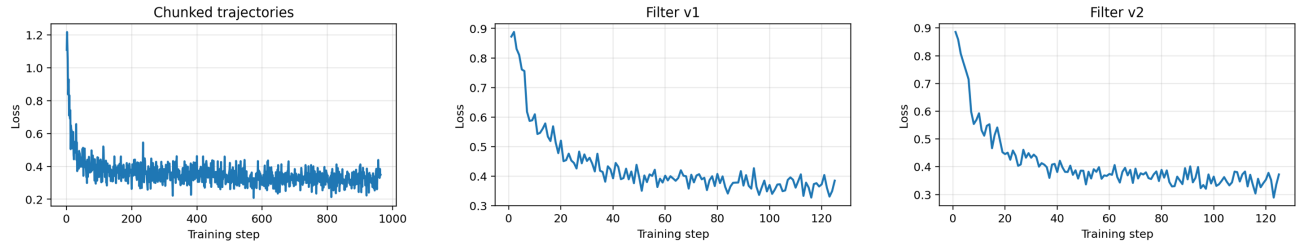
Table 1: Trajectory length statistics for the four preprocessing views. Raw full trajectories are shown for reference; the other three views were used for finetuning and evaluation.

Method	Mean	Median	Max	95th pct.	Comment
Raw full trajectories	46,801.0	43,394.0	257,031	75,272.2	Too long for stable small-model SFT
Filter v1	25,844.0	23,313.0	110,740	44,736.3	Keeps local progress-making steps
Filter v2	32,291.0	29,454.0	131,224	57,466.2	Adds look-ahead progress criterion
Chunked	6,516.0	6,002.0	16,380	11,092.8	Prompt-completion windows

Table 2: Held-out evaluation on the 100-trajectory test split. Exact-match metrics are intentionally strict and should be read as diagnostics, not as end-to-end task success.

Method	Init. loss	Final loss	Loss red.	PPL	Tool acc.	Arg. EM	Joint EM	Finish acc.
Filter v1	0.8722	0.3844	55.9%	1.4500	61.3%	16.7%	16.7%	0.0%
Filter v2	0.8856	0.3721	58.0%	1.4224	63.1%	17.4%	17.4%	0.0%
Chunked	1.1094	0.3493	68.5%	1.3362	70.2%	22.9%	22.7%	0.0%

All three runs reported 100 test trajectories. The filtered runs produced 6,212 evaluated steps and 912 truncated steps. The chunked run produced 6,207 evaluated steps and only 5 truncated steps.



(a) Chunked trajectories (b) Filter v1 (c) Filter v2

Figure 2: Training loss curves for the three preprocessing variants. Chunking runs for more steps because it produces many more supervised windows, but it also shows the strongest overall loss reduction.

Finally, our current evaluation happens offline. We measure imitation quality on static trajectories rather than actual task completion. This makes the results useful for preprocessing research, but not yet for claiming agent competence.

10 Conclusion

We studied trajectory distillation for SWE agents using `Nemotron-SWE-v1` and a 4B student model. Among the three preprocessing strategies we tested, chunking into prompt-completion windows gave the best trade-off between length reduction and retained supervision. It achieved the best perplexity and exact-match metrics, while the two filtering strategies remained useful but weaker alternatives.

The broader lesson is that distillation for software engineering agents should preserve local decision structure rather than merely delete long parts of the trace. The next stage is to validate distilled policies in executable sandboxes, where success is measured by task completion instead of token-level agreement.

References

[1] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741*.

[2] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. 2024. Training Software Engineering Agents and Verifiers with SWE-Gym. *arXiv preprint arXiv:2412.21139*.

[3] Minki Kang, Jongwon Jeong, Seanie Lee, Jaewoong Cho, and Sung Ju Hwang. 2025. Distilling LLM Agent into Small Models with Retrieval and Code Tools. *arXiv preprint arXiv:2505.17612*.

[4] Zonghan Yang, Shengjie Wang, Kelin Fu, Wenyang He, Weimin Xiong, Yibo Liu, Yibo Miao, Bofei Gao, Yejie Wang, Yingwei Ma, Yanhao Li, Yue Liu, Zhenxing Hu, Kaitai Zhang, Shuyi Wang, Huarong Chen, Flood Sung, Yang Liu, Yang Gao, Zhilin Yang, and Tianyu Liu. 2025. Kimi-Dev: Agentless Training as Skill Prior for SWE-Agents. *arXiv preprint arXiv:2509.23045*.